

V-Server Setup

A complete documentation of setting up a virtual server (V-Server), including SSH configuration, web server installation, and GitHub integration.

Table of Contents

1. [Generating SSH Keys](#)
 2. [First Login to the Server](#)
 3. [Registering an SSH Key on the Server](#)
 4. [Disabling Password Login](#)
 5. [Installing and Configuring nginx](#)
 6. [Setting Up an Alternative nginx Configuration](#)
 7. [Creating SSH Aliases](#)
 8. [Setting Up GitHub on the V-Server](#)
-

1. Generating SSH Keys

First, check whether SSH keys already exist on your local machine (the tilde `~` is a shorthand for the current user's home directory):

```
ls ~/.ssh
```

If no key exists, generate a new one:

```
ssh-keygen -t ed25519
```

This creates two files — the one ending in `.pub` is your **public key**.

To read the public key:

```
cat ~/.ssh/id_ed25519.pub
```

To create an additional key in a custom directory:

```
mkdir -p ~/.ssh/my-server  
ssh-keygen -t ed25519 -f ~/.ssh/my-server/server_ed25519
```

Read the new public key:

```
cat ~/.ssh/my-server/server_ed25519.pub
```

2. First Login to the Server

If your public key was registered during server creation (e.g. via your hosting provider's dashboard), you can log in directly:

```
ssh root@<server-ip>
```

3. Registering an SSH Key on the Server

If the server was set up with password login (which is common with some providers), you need to manually register your public key. Run the following command from your **local machine** (not from within the server session):

```
ssh-copy-id -i ~/.ssh/my-server/server_ed25519.pub username@<server-ip>
```

This command:

1. Reads the public key from the specified path on your local machine
2. Connects to the server using password authentication
3. Appends the public key to `~/.ssh/authorized_keys` on the server

After this, you can log in without a password:

```
ssh -i ~/.ssh/my-server/server_ed25519 username@<server-ip>
```

4. Disabling Password Login

To protect the server against brute-force attacks, it is recommended to disable password-based SSH login once key-based authentication is working.

Open the SSH server configuration file on the **server**:

```
sudo nano /etc/ssh/sshd_config
```

Find the line `#PasswordAuthentication yes`, uncomment it, and change `yes` to `no`:

```
PasswordAuthentication no
```

Then restart the SSH service to apply the changes:

```
sudo systemctl restart ssh
```

 **Important:** Make sure your SSH key login works before disabling password login, or you may lock yourself out.

Verifying Password Login is Disabled

To confirm that password login is actually disabled, run the following command from your local machine:

```
ssh -o PubkeyAuthentication=no username@<server-ip>
```

If you see **Permission denied (publickey)** without being prompted for a password, password login is successfully disabled.

5. Installing and Configuring nginx

To serve files over the web, install **nginx** as the web server:

```
sudo apt update          # Update the package manager
sudo apt install nginx -y # Install nginx
```

Check the nginx service status:

```
systemctl status nginx.service
```

Once running, nginx's default welcome page is accessible at:

```
http://<server-ip>/
```

6. Setting Up an Alternative nginx Configuration

To serve a custom page on a different port, follow these steps:

Create a new directory for the alternative site:

```
sudo mkdir /var/www/alternative
```

Create a new HTML file:

```
sudo touch /var/www/alternative/index.html
```

Create a new nginx configuration:

```
sudo nano /etc/nginx/sites-enabled/alternative
```

Add the following configuration block:

```
server {  
    listen 8081;  
    listen [::]:8081;  
  
    root /var/www/alternative;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}
```

Configuration explained:

Directive	Description
<code>listen 8081</code>	Listens on port 8081 for IPv4 connections
<code>listen [::]:8081</code>	Listens on port 8081 for IPv6 connections
<code>root</code>	Directory from which files are served
<code>index</code>	Default file opened when accessing the root URL
<code>try_files \$uri</code>	Checks if the requested file exists directly
<code>try_files \$uri/</code>	Checks if a matching directory exists
<code>=404</code>	Returns a 404 error if neither is found

Add content to the HTML file:

```
sudo nano /var/www/alternative/index.html
```

Check the nginx configuration for syntax errors:

```
sudo nginx -t
```

If the firewall is active, allow port 8081:

```
sudo ufw allow 8081
```

Restart nginx to apply the new configuration:

```
sudo systemctl restart nginx
```

Both configurations now run simultaneously on different ports:

URL	Description
<a href="http://<server-ip>/">http://<server-ip>/	Default nginx page (Port 80)
<a href="http://<server-ip>:8081/">http://<server-ip>:8081/	Your alternative page (Port 8081)

7. Creating SSH Aliases

The full login command can get quite long. You can create an alias to shorten it. The SSH config file (`~/.ssh/config`) provides a persistent and flexible way to manage SSH connections.

View or create the config file:

```
nano ~/.ssh/config
```

Basic entry — using IP address directly:

```
Host <server-ip>
  User username
  PreferredAuthentications publickey
  IdentityFile ~/.ssh/my-server/server_ed25519
```

With this, `ssh <server-ip>` is sufficient to connect.

Using a custom alias instead:

```
Host my-server
  HostName <server-ip>
  User username
  PreferredAuthentications publickey
  IdentityFile ~/.ssh/my-server/server_ed25519
```

Now you can simply run:

```
ssh my-server
```

8. Setting Up GitHub on the V-Server

To enable the server to communicate directly with GitHub (e.g. for cloning private repositories), generate and register a dedicated SSH key **on the server**.

1. Log in to the server and generate an SSH key:

```
ssh my-server
ssh-keygen -t ed25519 -C "your-email@example.com"
```

Display and copy the public key:

```
cat ~/.ssh/id_ed25519.pub
```

2. Register the public key on GitHub:

1. Go to github.com → Profile picture → **Settings**
2. In the left menu: **SSH and GPG keys**
3. Click **New SSH key**
4. Enter a descriptive name (e.g. **vserver**) and paste the public key
5. Confirm with **Add SSH key**

3. Test the connection:

```
ssh -T git@github.com
```

A successful response looks like:

```
Hi <username>! You've successfully authenticated...
```

4. Configure Git on the server:

```
git config --global user.name "your-github-username"  
git config --global user.email "your-email@example.com"
```

5. Clone a repository:

```
git clone git@github.com:<username>/<repository>.git
```